



CYBER SECURITY AND AI CASE STUDIES

MOD006570

PROJECT LOGBOOK

**Intrusion Detection and Insider Label Poisoning
using the UNSW-NB15 Dataset**

Student ID: 2454205

GitHub Link: <https://github.com/vinaykoparthy-cpu/Cyber-Security.git>

Session 1 — Understanding the assignment

Focus: Converting the brief into a useful inquiry

Working file: `unsw_intrusion_poisoning_study.ipynb`

What I worked on

After carefully reading the brief, divide the work into Parts A, B, and C.

- Identified the essential query: to what extent might label poisoning impair a neural network's ability to identify attacks?
- Decided that in order to evaluate each model using the same clean test set, the official test labels must remain unaltered.
- Documented the anticipated results: secure agent architecture, poisoning experiment, model comparison, EDA, and critical discussion.

Project route

Assignment section	Work recorded
Part A	Examine UNSW-NB15, contrast models, and assess the clean detector.
Part B	Model both targeted and random training-label poisoning.
Part C	Create a safe process for deploying autonomous agents.
Evidence	includes the final report, stored figures, results tables, and a Jupyter notebook.

What I Discovered

Achieving a high accuracy score is not the main goal of the evaluation. When someone in the training process modifies labels that appear genuine, it also raises questions about the model's reliability. As a result, my evaluation now focuses on attack recall, miss rate, and family-level conduct instead of just accuracy.

The following action

Verify the target labels, load the two official CSV files, and assess their quality.

Session 2: UNSW-NB15 data loading and verification

Files: UNSW_NB15_training-set.csv and UNSW_NB15_testing-set.csv

Tools: Python, pandas and Jupyter Notebook

What I worked on

- The original training and testing filenames were loaded without being renamed.
- The notebook's personal computer paths were eliminated in favour of a straightforward data folder placement.
- Verified the binary target, duplicate rows, columns, row counts, and missing values.
- Verified that label 1 denotes attack traffic and label 0 denotes regular traffic.

Audit of datasets

File	Rows	Columns	Missing cells	Duplicate rows
Training set	175,341	45	0	0
Testing set	82,332	45	0	0

Issue and solution

Due to an unintentional trailing space in the folder name, the initial file check was unsuccessful. In order to prevent the display of a personal login or the entire computer path, I fixed the folder reference and maintained the general error message.

What I Discovered

Although a clean schema check is crucial, it cannot demonstrate the reliability of the labels. Later, the poisoning experiment modifies the meaning of some training records while maintaining the validity of every value.

Verified Source

Official dataset page: [UNSW-NB15 Dataset](#)

Session 3 :Analysis of exploratory data

Focus: Appropriate traffic patterns and class balance

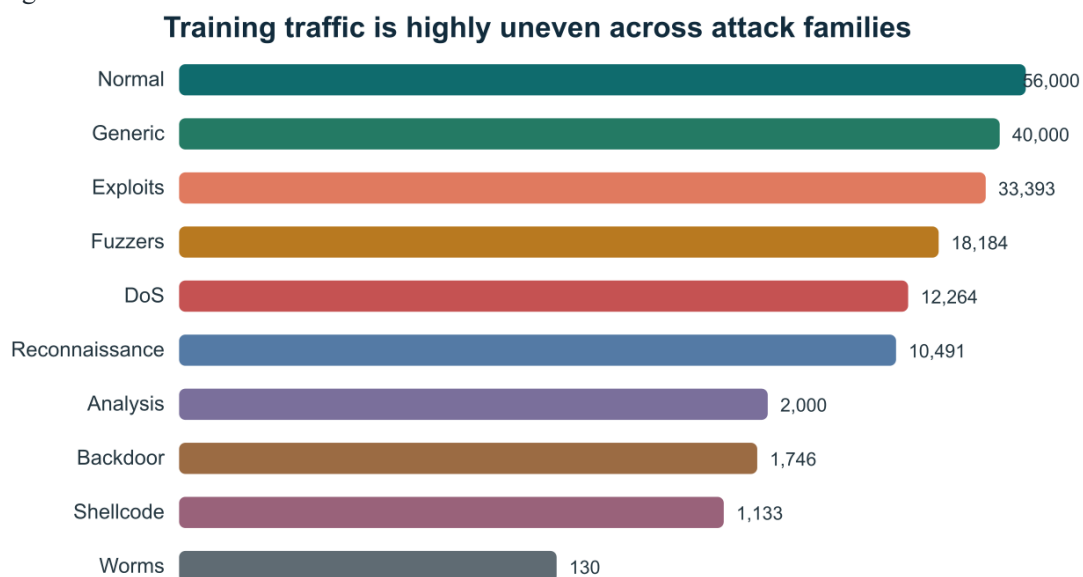
Evidence: Attack dispersion and thorough traffic summaries serve as proof.

What I worked on

- Examined the training data's nine attack families against regular traffic.
- Because the rare classes vanished next to the largest groupings on a standard bar chart, a logarithmic scale was used.
- Rather than depending just on means, robust summaries were employed and skewed byte, packet, and duration variables were checked.
- Before selecting a model, the training and test distributions were compared.

The primary finding

The dataset is dominated by Normal, Generic, and Exploits, whereas Worms only includes 130 training records. Therefore, even if the detector fails on a rare attack family, a single accuracy value may appear strong.



Flow count shown on a logarithmic horizontal scale

Evidence 1. Training-flow distribution by traffic category on a logarithmic scale.

What I Discovered

The EDA justified reporting attack recall and recall by attack family. It also suggested that several Reporting attack recall and recall by attack family was justified by the EDA. Additionally, it indicated that because malicious and regular flows overlap in basic byte-based views, several aspects must be considered simultaneously.

The following action

Create a single, standardised preprocessing pipeline and contrast many neural networks using a linear baseline.

Session 4: Model selection and preprocessing

Focus: Neural-network and baseline models are fairly compared

Selection rule: Highest validation attack F1

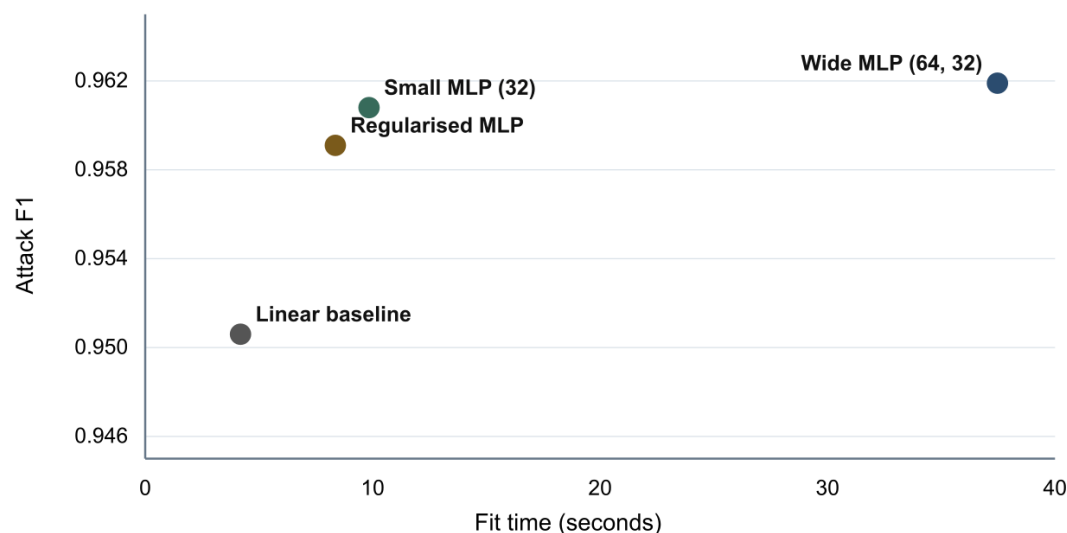
What I worked on

- Numerical and category data were kept apart inside a single, reusable preprocessing pipeline.
- One-hot encoded categorical values and standardised and imputed numerical values.
- Small, Wide, and Regularised MLP models were compared to class-weighted logistic regression.
- To maintain fairness in the comparison, the same split, preprocessing, and random seed were used.

Result

The maximum validation attack F1 of 0.9619 was attained by the Wide MLP with hidden layers of 64 and 32 units. Its ROC AUC was 0.9904 and its balanced accuracy was 0.9325. I saw the Small MLP as a viable option for a limited gateway because it trained more quickly and was only 0.0011 lower in F1.

Validation F1 versus training cost



Evidence 2. Validation attack F1 compared with model fitting time.

What I Discovered

It is not advisable to select a neural network merely because it seems more sophisticated. The validation findings provided a quantified justification for choosing the Wide MLP, while the baseline demonstrated what a straightforward linear boundary could already do.

Session 5: Evaluating the clean intrusion detector

Model: Wide MLP (64, 32)

Test policy: One last assessment of the unaltered official test set

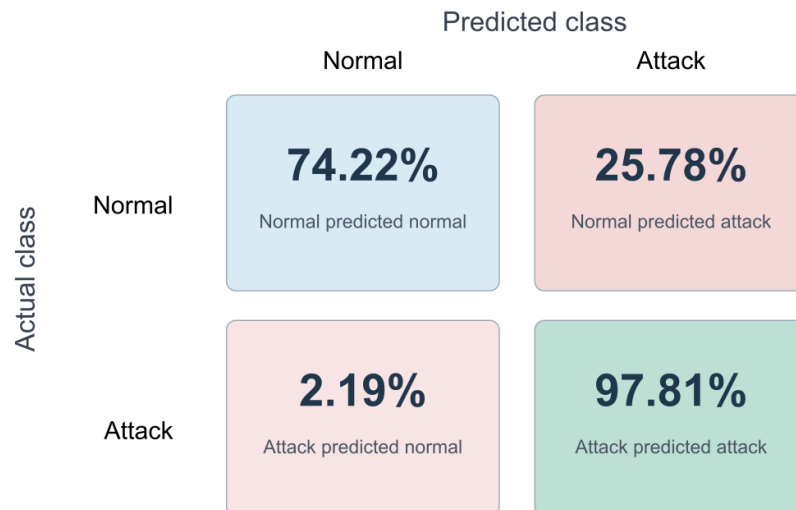
What I worked on

- All 175,341 training flows had the chosen pipeline refitted.
- Determined the confusion matrix, ROC AUC, precision, recall, F1, and balanced accuracy.
- Rather than just displaying overall accuracy, it interpreted both missed assaults and false alarms.

Result

Attack precision of 0.8229, recall of 0.9781, F1 of 0.8938, balanced accuracy of 0.8601, and ROC AUC of 0.9783 were all achieved by the clean model. While 25.78% of regular flows were mistakenly reported, it identified 97.81% of attacks. Although the recall was robust, a true IoT implementation would require calibration of the false-alarm rate.

Clean model: row-normalised confusion matrix



Evidence 3. Row-normalised confusion matrix for the clean Wide MLP.

What I Discovered

The detector is not always operationally safe just because it has a high attack recall. An analyst may get overburdened or needless automated blocking may result from an excessive number of erroneous signals.

Next step

measure how the clean-test behaviour changes by retraining the same pipeline following controlled label changes.

Session 6: Creating the label-poisoning

Threat actor: An insider with the ability to modify training labels but not features or test label

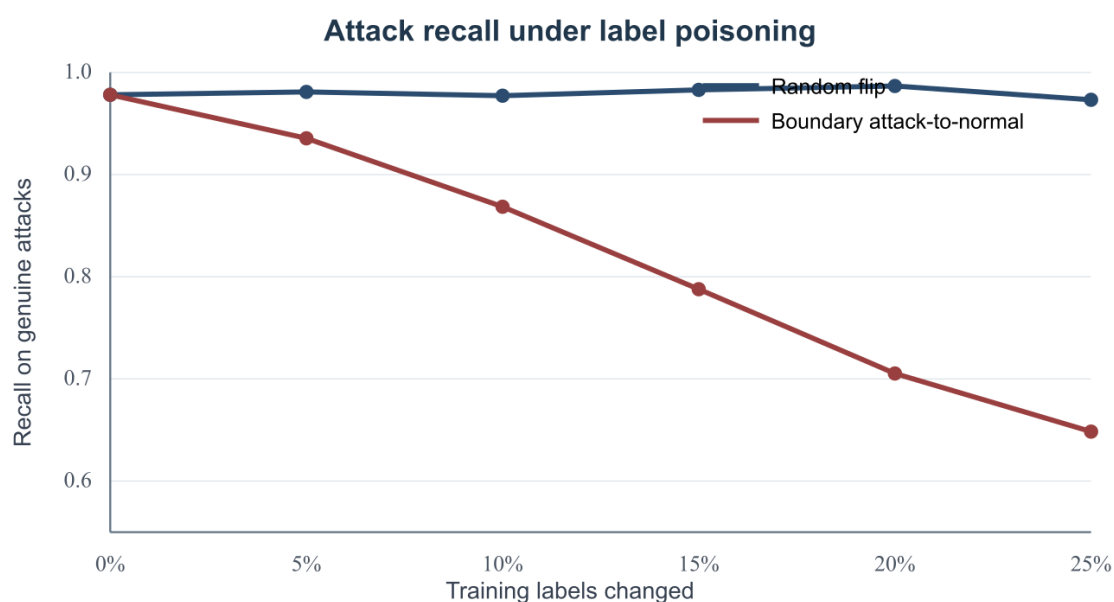
Budgets: 0%, 5%, 10%, 15%, 20% and 25% of the training set

What I worked on

- As a universal label-noise baseline, a seeded random-flip technique was developed.
- Developed a focused approach that modifies low-confidence attack labels from attack to normal.
- At each poisoning level, a new copy of the Wide MLP was retrained.
- Every poisoned model was assessed using the same unaltered test set.

Why the targeted approach differs

A portion of Random Flipping's budget is used to turn typical records into attacks. The targeted method focuses on boundary situations that the clean model already deems questionable, using each adjustment to conceal a real attack. The comparison was fair by budget since both approaches altered 8,767 labels at 5%.



Evidence 4. Attack recall after random and boundary-targeted label poisoning.

What I Discovered

The test set needs to be kept tidy. Test label changes would merely facilitate the examination; they wouldn't determine whether poisoning harmed the trained detector.

Session 7 Examining the effects of poisoning

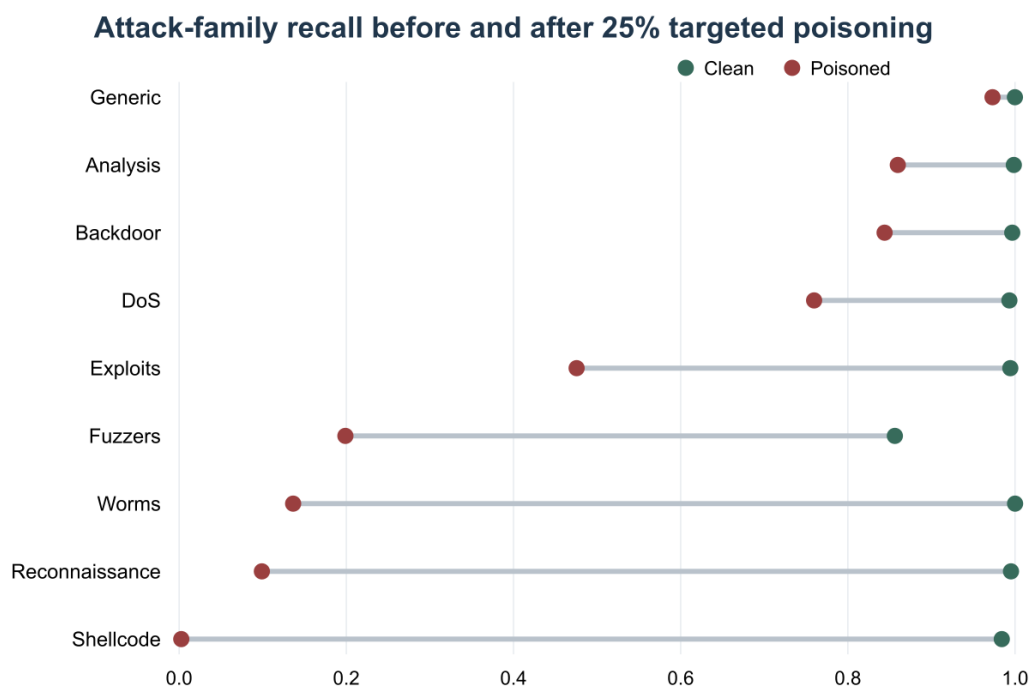
Focus: Memory loss, covert assaults, and consequences at the family level

Worst tested case: 25% boundary-targeted poisoning

Result

Although it raised false alarms, random flipping left attack recall ranged from 0.9732 to 0.9867. Recall dropped to 0.9355 at 5%, 0.8685 at 10%, and 0.6484 at 25% due to targeted poisoning, which was far more harmful. 35.16% of real attacks were missed by the final model, or around 15,939 attacks as opposed to 993 for the clean model.

- The recall of shellcode decreased from 0.9841 to 0.0026.
- The recall for reconnaissance decreased from 0.9951 to 0.0990.
- The recall of worms decreased from 1.0000 to 0.1364.
- The recall of fuzzers decreased from 0.8562 to 0.1989.



Evidence 5. Clean and poisoned recall for each attack family.

Interpretation

Under intensive targeted poisoning, precision increased and false alarms decreased, but this was not an improvement. Because many attacks were silently accepted as normal, the model raised less alerts. Therefore, a weakened model might appear healthier on a dashboard that only monitored precision or alert volume.

What I Discovered

The elimination of entire threat behaviours can be concealed by global metrics. Therefore, a deployment gate should display the support count behind each rate and specify minimal recall levels for specific attack families.

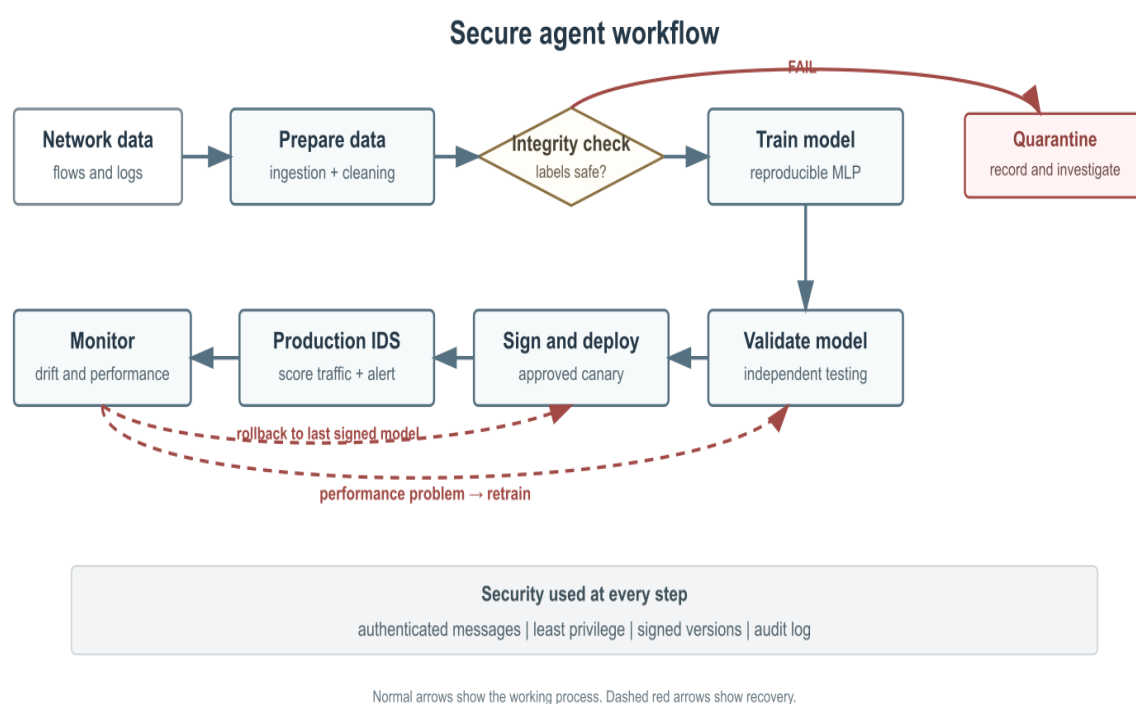
Session 8: Final submission and secure architecture

Focus: Converting the experiment into useful security measures

Final results: Notebook, data, findings, report, and project journal

What I worked on

- Separate roles were created for ingestion, integrity, training, validation, signing, deployment, and monitoring.
- Used least privilege to prevent a training component from approving its own model and changing labels.
- Before a candidate can be signed, the pipeline, model, and dataset hashes must be obtained.
- Rollback to the last trusted model, canary deployment, credential revocation, and quarantine were added.



Evidence 6. Simple secure workflow for training, validating and deploying the intrusion detector.

Concluding thoughts

The most significant outcome was not the high recall of the clean model, but rather the speed at which a targeted insider could undermine that security while leaving behind data that appeared legitimate. This made a connection between cybersecurity practice and modelling work. The AI system includes integrity controls, independent validation, and recovery strategies; they are not after-training add-ons.